

■ LEVEL 12 • RECURSION

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	The problem becomes the same problem on a smaller input. Identify base case, smaller problem, and progress.
Reference Examples	Factorial • Fibonacci • Tree Traversals • Merge Sort • Power(x,n)
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 12 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Factorial	Recursion	Absent
2	Fibonacci Number	Recursion / DP	#509
3	Power of Two	Recursion / Division	#231
4	Pow(x, n)	Divide and Conquer Recursion	#50
5	Reverse String	Recursion + Two Pointers	#344
6	Binary Tree Preorder Traversal	Tree Recursion	#144
7	Fibonacci Number	Recursion / DP contrast	509
8	Pow(x, n)	Divide-and-Conquer Recursion	50
9	K-th Symbol in Grammar	Recursive Halving	779
10	Reverse Linked List Recursively	Recursive Pointer Rewiring	206

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Factorial

Pattern: Recursion

LeetCode: ■ Absent

■ Input	integer n
■ Output	n!
■ Constraints	$n \geq 0$
■ Main Idea / Pattern	Base case $n=0$. Otherwise $n! = n \times (n-1)!$
■■ Time Complexity	$O(n)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
long long factorial(int n) {  
    if (n == 0)  
        return 1;  
    return 1LL * n * factorial(n - 1);  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Fibonacci Number

Pattern: Recursion / DP

LeetCode: #509

■ Input	integer n
■ Output	nth Fibonacci number
■ Constraints	$0 \leq n \leq 30$
■ Main Idea / Pattern	Base cases 0 and 1. Recursively solve n-1 and n-2. This is the direct recursive form.
■ Time Complexity	$O(2^n)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
int fib(int n) {  
    if (n <= 1)  
        return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Power of Two

Pattern: Recursion / Division

LeetCode: #231

■ Input	integer n
■ Output	whether n is a power of two
■ Constraints	32-bit integer
■ Main Idea / Pattern	A positive power of two repeatedly divides by 2 until it reaches 1.
■■ Time Complexity	$O(\log n)$
■ Space Complexity	$O(\log n)$

■ Optimal C++ Code

```
bool isPowerOfTwo(int n) {  
    if (n == 1) return true;  
    if (n <= 0 || n % 2 != 0) return false;  
    return isPowerOfTwo(n / 2);  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Pow(x, n)

Pattern: Divide and Conquer Recursion

LeetCode: #50

■ Input	real x and integer n
■ Output	x raised to n
■ Constraints	n may be negative
■ Main Idea / Pattern	Exponentiation by squaring reduces the exponent by half each recursive call.
■■ Time Complexity	$O(\log n)$
■ Space Complexity	$O(\log n)$

■ Optimal C++ Code

```
double myPow(double x, long long n) {  
    if (n == 0) return 1.0;  
  
    double half = myPow(x, n / 2);  
  
    if (n % 2 == 0)  
        return half * half;  
  
    if (n > 0)  
        return half * half * x;  
  
    return half * half / x;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Reverse String

Pattern: Recursion + Two Pointers

LeetCode: #344

■ Input	character array
■ Output	characters reversed in-place
■ Constraints	$1 \leq n$
■ Main Idea / Pattern	Swap the two ends, then recurse inward.
■ Time Complexity	$O(n)$
■ Space Complexity	$O(n)$

■ Optimal C++ Code

```
void reverseString(vector<char>& s, int l, int r) {
    if (l >= r)
        return;

    swap(s[l], s[r]);
    reverseString(s, l + 1, r - 1);
}

void reverseString(vector<char>& s) {
    reverseString(s, 0, s.size() - 1);
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 Binary Tree Preorder Traversal

Pattern: Tree Recursion

LeetCode: #144

■ Input	binary tree root
■ Output	preorder values
■ Constraints	binary tree
■ Main Idea / Pattern	Visit root, recursively traverse left, then right.
■■ Time Complexity	O(n)
■ Space Complexity	O(h)

■ Optimal C++ Code

```
void dfs(TreeNode* root, vector<int>& ans) {
    if (!root) return;
    ans.push_back(root->val);
    dfs(root->left, ans);
    dfs(root->right, ans);
}

vector<int> preorderTraversal(TreeNode* root) {
    vector<int> ans;
    dfs(root, ans);
    return ans;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Fibonacci Number

Pattern: Recursion / DP contrast

LeetCode: 509

Input	n
Output	nth Fibonacci number
Constraints	0 <= n <= 30
Main Idea / Pattern	Direct recursion demonstrates the repeated-subproblem structure; memoization removes duplicate work.
Time Complexity	O(2^n) direct recursion
Space Complexity	O(n) call stack

Optimal C++ Code

```
class Solution {  
public:  
    int fib(int n) {  
        if (n <= 1) return n;  
        return fib(n - 1) + fib(n - 2);  
    }  
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Pow(x, n)

Pattern: Divide-and-Conquer Recursion

LeetCode: 50

Input	x, n
Output	x raised to the power n
Constraints	$-2^{31} \leq n \leq 2^{31} - 1$
Main Idea / Pattern	Square the result for half the exponent; handle odd exponents and negative n.
Time Complexity	$O(\log n)$
Space Complexity	$O(\log n)$

Optimal C++ Code

```
class Solution {
    double fast(double x, long long n) {
        if (n == 0) return 1.0;
        double half = fast(x, n / 2);
        if (n % 2 == 0) return half * half;
        return n > 0 ? half * half * x : half * half / x;
    }
public:
    double myPow(double x, int n) {
        return fast(x, (long long)n);
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 K-th Symbol in Grammar

Pattern: Recursive Halving

LeetCode: 779

Input	n, k
Output	k-th symbol in row n
Constraints	1 <= n <= 30
Main Idea / Pattern	The second half mirrors the first with bits flipped, so map k to its parent.
Time Complexity	O(n)
Space Complexity	O(n)

Optimal C++ Code

```
class Solution {
public:
    int kthGrammar(int n, int k) {
        if (n == 1) return 0;
        int parent = kthGrammar(n - 1, (k + 1) / 2);
        if (k % 2 == 1) return parent;
        return 1 - parent;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Reverse Linked List Recursively

Pattern: Recursive Pointer Rewiring

LeetCode: 206

Input	head
Output	reversed linked list
Constraints	0 <= nodes <= 5000
Main Idea / Pattern	Reverse the smaller list first, then attach the current node after its successor.
Time Complexity	O(n)
Space Complexity	O(n) call stack

Optimal C++ Code

```
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if (!head || !head->next) return head;
        ListNode* newHead = reverseList(head->next);
        head->next->next = head;
        head->next = nullptr;
        return newHead;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.